

R2 — O espaço de dados, em Python

A primeira camada do simulador, e os testes como especificação

Microcontroladores — DENE/UFMT

OBJETIVOS

- Implementar o espaço de dados do PIC18F4550 conforme a especificação do encontro 1.
- Reproduzir o comportamento da região não implementada, em vez de rejeitá-la.
- Resolver o campo `f` de oito bits num endereço de doze, com e sem BSR.
- Escrever um teste que distinga a implementação correta de uma que apenas parece correta.

Esta sessão é no computador, não na bancada. Não há kit, não há osciloscópio e não há risco elétrico — o que não a torna menos laboratório: vocês vão construir o instrumento com que os próximos encontros vão medir coisas.

NOTA

É também a primeira sessão com nota. A folha de previsão da aula 1 é entregue no início.

1 Por que escrever o simulador

O encontro 1 especificou um espaço de memória: 4096 endereços, uma região que não existe, e um atalho chamado `access bank`. Especificação é fácil de ler e difícil de ter certeza que se entendeu.

Escrever o simulador é a prova de que a leitura foi correta. Não é possível implementar `le(0x900)` sem ter decidido o que ele devolve, e não é possível implementar `endereco_efetivo(0x8C, 0)` sem ter entendido por que a fronteira do `access bank` fica em `0x5F` e não em `0x7F`.

CONCEITO

A especificação vira executável. Se ela estava ambígua na sua cabeça, o interpretador descobre.

E há um segundo motivo, que só aparece no encontro 7. Um simulador é onde se estuda **algoritmo** — determinístico, repetível, com o tempo parado. A bancada é onde se estuda **fenômeno**.

NOTA

Cada ferramenta serve para a metade que ela faz bem, e saber onde o simulador mente é parte do conteúdo. Hoje vocês estão do lado de dentro dessa mentira: escolhendo o que ela vai reproduzir e o que não vai.

2 A especificação

Da folha de dados, seção de organização da memória, e do encontro 1:

0x000–0x7FF	RAM de uso geral, bancos 0 a 7
0x800–0xEFF	Não implementada. Leitura devolve 0, escrita é descartada
0xF00–0xFFF	Registradores de função especial

E a resolução do campo de endereço da instrução, que tem oito bits:

a = 1	endereço = (BSR << 8) f — o banco vem do BSR
a = 0	access bank: <code>f</code> de <code>0x00</code> a <code>0x5F</code> vai para <code>0x000–0x05F</code> ; <code>f</code> de <code>0x60</code> a <code>0xFF</code> vai para <code>0xF60–0xFFF</code>

A fronteira do `access bank` é `0x5F/0x60`, e não `0x7F/0x80`.

ATENÇÃO

São 96 bytes de RAM e 160 SFRs, e não 128 de cada. A janela de 256 endereços é montada com dois pedaços que **não são vizinhos**, e essa descontinuidade é o detalhe que separa uma implementação correta de uma que passa nos testes fáceis.

3 O esqueleto e os testes

Vocês recebem três arquivos:

<code>memoria.py</code>	O esqueleto, com os métodos a completar marcados
<code>teste_memoria.py</code>	Os testes — a especificação, executável
—	O gabarito não é distribuído hoje

```
python teste_memoria.py
```

Leia os testes antes de escrever qualquer código. Cada um deles traz, entre parênteses, a regra de onde veio — “não existe endereço inválido, existe truncado”, “96 bytes de RAM e 160 SFRs, não 128 e 128”.

CONCEITO

Os testes não são conferência posterior: eles **são** o enunciado. Um requisito que você não consegue enunciar como teste é um requisito que você ainda não entendeu.

4 Parte 1 — o endereço tem doze bits

Implemente `le` e `escreve` de modo que a Parte 1 dos testes passe.

TAREFA

Três comportamentos: guardar e devolver; truncar o endereço em doze bits; truncar o valor em oito.

O teste do endereço de treze bits costuma provocar a reação errada: levantar exceção.

ATENÇÃO

O chip não levanta nada. Ele tem doze fios de endereço, e o décimo terceiro bit simplesmente não chega a lugar nenhum. Um simulador que rejeita `0x1123` está sendo mais rigoroso que o silício — e essa gentileza esconde exatamente a classe de defeito que ele deveria expor.

5 Parte 2 — a região que lê zero

Faça a Parte 2 passar. Leitura em `0x800–0xEFF` devolve 0; escrita ali é descartada em silêncio.

TAREFA

Repare que os testes verificam as **duas pontas** da região, e também os dois endereços imediatamente fora dela.

Por que testar as pontas: o erro clássico é usar `<` onde cabia `<=`, e ele não aparece no meio da faixa. Um teste em `0x900` passa com a implementação errada; um teste em `0xEFF` não.

NOTA

Esse hábito — testar a fronteira, não o meio — vale para o resto da vida profissional de vocês.

6 Parte 3 — o campo `f` e o access bank

Implemente `endereco_efetivo(f, a)`.

TAREFA

Comece pelo caso `a = 1`, que é uma concatenação. Depois faça o access bank, e preste atenção na fronteira.

Dois dos testes desta parte estão marcados com **(P5)**, e eles respondem à previsão que vocês trouxeram: **que teste prova que a sua implementação não é apenas “somar `0xF00` quando `f` é grande”?**

CONCEITO

O primeiro chama `endereco_efetivo(0x8C, 0)` com três valores diferentes de BSR e exige que os três resultados sejam iguais — se o BSR estivesse sendo usado, eles seriam diferentes.

O segundo chama `endereco_efetivo(0x12, 0)` e exige `0x012` — “somar `0xF00` sempre” daria `0xF12`.

Um teste que passa com a implementação errada não é um teste. Estes dois foram construídos olhando para as implementações erradas mais prováveis, e essa é a técnica que a Parte 4 vai pedir de vocês.

7 Parte 4 — o teste que você escreve

Os testes distribuídos cobrem a especificação. Escreva **mais dois**, e para cada um diga qual implementação errada ele mata.

TAREFA

Sugestões do que ainda não está coberto: o valor inicial de todas as células; o que acontece quando o BSR passa de quinze; a diferença entre escrever pelo endereço e escrever pelo campo `f`; a independência entre células vizinhas.

8 Entrega

Entregue `memoria.py` completo, com os dois testes acrescentados ao final de `teste_memoria.py`, e uma página respondendo:

TAREFA

(a) Você guardou 4096 posições ou 2048? Justifique pela especificação — e diga o que a outra escolha teria custado em código.

(b) Qual dos testes distribuídos você quebrou mais vezes durante a implementação? O que ele estava tentando te dizer?

(c) Os dois testes que você escreveu, com o nome da implementação errada que cada um mata.

No R3: o LED, o atraso e a frequência, com o osciloscópio.

NOTA

O simulador de hoje guarda valores e não sabe que horas são. Na semana que vem vocês vão medir, na placa, o tempo que o encontro 0 mandou vocês chutarem — e descobrir se o oscilador é mesmo o que dizem que é.